

Exercises

Design principles & Patterns

Exercise 1: Implementing the Singleton Pattern

```
Singleton.java
import java.util.*;
class Singleton {
    private static Singleton instance;
    Singleton() {
        System.out.println("Created a Singleton Class");
    } //Private constructor
    public static Singleton getInstance(){
        if (instance==null){
            instance=new Singleton();
        }
        return instance;
    }
}
```

PORTS

▼ TERMINAL

```
● PS P:\Cognizant_exercise> javac Singleton.java Main.java
● PS P:\Cognizant_exercise> java Main
Created a Singleton Class
Both are same
❖ PS P:\Cognizant_exercise> █
```

Main_ Loading...

```
1 public class Main{
2     public static void main(String args[]){
3         Singleton s1=Singleton.getInstance();
4         Singleton s2=Singleton.getInstance();
5         if(s1==s2){
6             System.out.println("Both are same");
7         }
8     }
9 }
```

Exercise 2: Implementing the Factory Method Pattern

```
FactoryMethod.java
1  import java.util.*;
2  interface Animal{
3      void speak();
4  }
5  class Dog implements Animal{
6      public void speak(){
7          System.out.println("Woof");
8      }
9  }
10 class Cat implements Animal{
11     public void speak(){
12         System.out.println("Meow");
13     }
14 }
15 class FactoryMethod {
16     public static void getFactory(String type){
17         if(type=="Dog"){
18             Dog d=new Dog();
19         }
20         else if(type=="Cat"){
21             Cat c=new Cat();
22         }
23         else{
24             System.out.println("Invalid Type");
25         }
26     }
27 }
```

```
public class Main{  
    public static void main(String args[]){x:  
        Animal a=new Dog();  
        a.speak();  
        Animal b=new Cat();  
        b.speak();  
    }  
}
```

AL PORTS QUERY RESULTS

✓ **TERMINAL**

```
● PS P:\Cognizant_exercise> javac FactoryMethod.java Main.java  
● PS P:\Cognizant_exercise> java Main  
Woof  
Meow  
❖ PS P:\Cognizant_exercise> 
```

Exercise 2: E-commerce Platform Search Function

```
import java.util.*;

class Ecommerce {
    static class Product{
        int id;
        String pname;
        String Category;
        double price;
        Product(int id,String pname,String Category,double price){
            this.id=id;
            this.pname=pname;
            this.Category=Category;
            this.price=price;
        }
        public String toString(){
            return id + "-" + pname + "-" + Category + "Rs."+price;
        }
    }
    //It follows Interface Segregation Principle.
    interface SearchStrategy{
        List<Product> search(List<Product> products, String query);
    }
    //It follows Single Responsibility Principle as it has only one method to implement in the class which implements this
    static class KeywordSearch implements SearchStrategy{
```

```
        public List<Product> search(List<Product> products, String query){
            List<Product> result=new ArrayList<>();
            for(Product p:products){
                if(p.pname.toLowerCase().contains(query.toLowerCase())){
                    result.add(p);
                }
            }
            return result;
        }
    }
    static class PriceSearch implements SearchStrategy{
        public List<Product> search(List<Product> products, String query){
            List<Product> result=new ArrayList<>();
            double price=Double.parseDouble(query);
            for(Product p:products){
                if(p.price<=price){
                    result.add(p);
                }
            }
            return result;
        }
    }
}
```

```

}
static class ProductCatalog{
    private List<Product> products=new ArrayList<>();
    private Map<Integer,Product> productMap=new HashMap<>();
    void addProduct(Product p){
        products.add(p);
        productMap.put(p.id,p);
    }
    Product getById(int id){
        return productMap.get(id);
    }
    List<Product> search(SearchStrategy strategy,String query){
        return strategy.search(products,query);
    }
}

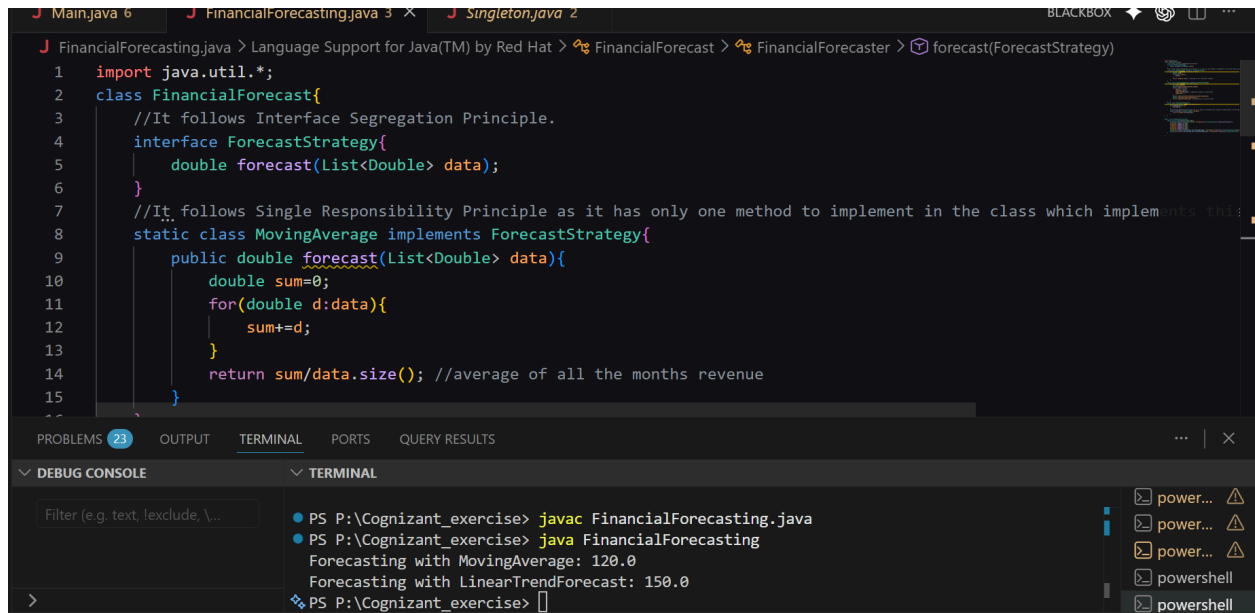
```

```

public class Main{
    Run main | Debug main | Run | Debug
    public static void main(String[] args){
        Ecommerce.ProductCatalog catalog=new Ecommerce.ProductCatalog();
        catalog.addProduct(new Ecommerce.Product(id: 1,pname: "Laptop",Category: "Electronics",price: 50000));
        catalog.addProduct(new Ecommerce.Product(id: 2,pname: "Mobile",Category: "Electronics",price: 20000));
        catalog.addProduct(new Ecommerce.Product(id: 3,pname: "Shirt",Category: "Clothing",price: 1000));
        catalog.addProduct(new Ecommerce.Product(id: 4,pname: "Jeans",Category: "Clothing",price: 2000));
        catalog.addProduct(new Ecommerce.Product(id: 5,pname: "Shoes",Category: "Footwear",price: 3000));
        System.out.println(x: "Search by keyword 'Laptop':");
        List<Ecommerce.Product> result=catalog.search(new Ecommerce.KeywordSearch(),query: "Laptop");
        for(Ecommerce.Product p:result){
            System.out.println(p);
        }
        System.out.println(x: "Search by price <= 2000:");
        result=catalog.search(new Ecommerce.PriceSearch(),query: "2000");
        for(Ecommerce.Product p:result){
            System.out.println(p);
        }
    }
}

```

Exercise 7: Financial Forecasting



```
1 import java.util.*;
2 class FinancialForecast{
3     //It follows Interface Segregation Principle.
4     interface ForecastStrategy{
5         double forecast(List<Double> data);
6     }
7     //It follows Single Responsibility Principle as it has only one method to implement in the class which implements this
8     static class MovingAverage implements ForecastStrategy{
9         public double forecast(List<Double> data){
10             double sum=0;
11             for(double d:data){
12                 sum+=d;
13             }
14             return sum/data.size(); //average of all the months revenue
15         }
16     }
17 }
```

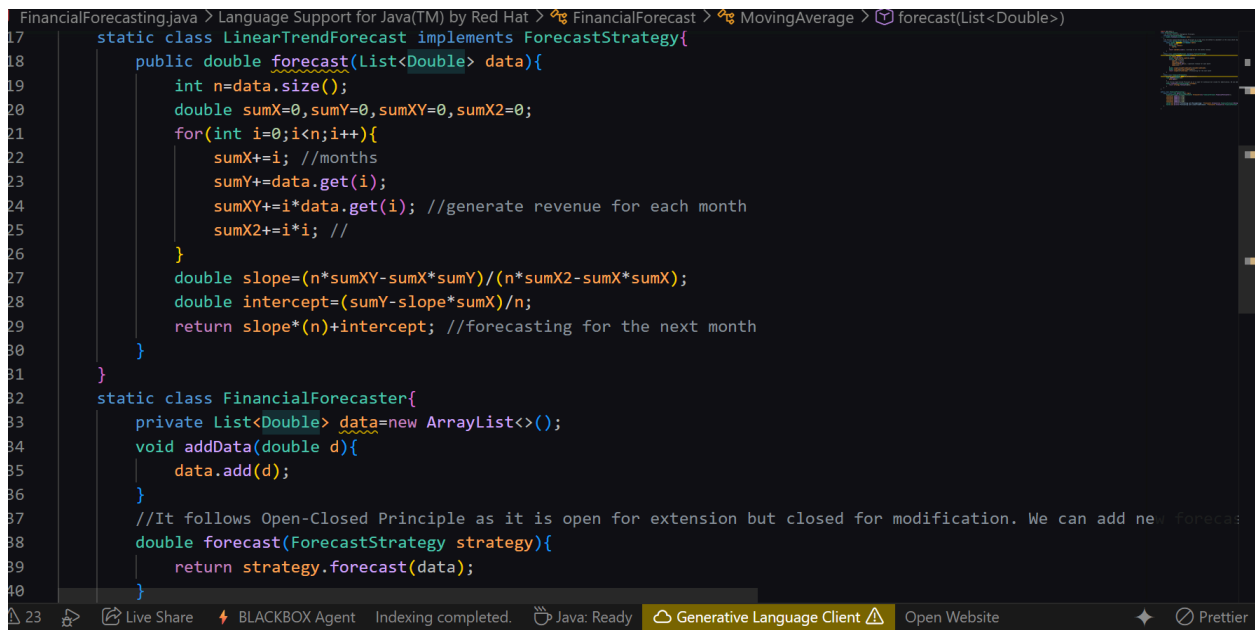
PROBLEMS 23 OUTPUT TERMINAL PORTS QUERY RESULTS

DEBBUG CONSOLE TERMINAL

Filter (e.g. text, exclude, ...)

- PS P:\Cognizant_exercise> javac FinancialForecasting.java
- PS P:\Cognizant_exercise> java FinancialForecasting
- Forecasting with MovingAverage: 120.0
- Forecasting with LinearTrendForecast: 150.0
- PS P:\Cognizant_exercise>

power... power... power... powershell powershell



```
17 static class LinearTrendForecast implements ForecastStrategy{
18     public double forecast(List<Double> data){
19         int n=data.size();
20         double sumX=0,sumY=0,sumXY=0,sumX2=0;
21         for(int i=0;i<n;i++){
22             sumX+=i; //months
23             sumY+=data.get(i);
24             sumXY+=i*data.get(i); //generate revenue for each month
25             sumX2+=i*i; //
26         }
27         double slope=(n*sumXY-sumX*sumY)/(n*sumX2-sumX*sumX);
28         double intercept=(sumY-slope*sumX)/n;
29         return slope*(n)+intercept; //forecasting for the next month
30     }
31 }
32 static class FinancialForecaster{
33     private List<Double> data=new ArrayList<>();
34     void addData(double d){
35         data.add(d);
36     }
37     //It follows Open-Closed Principle as it is open for extension but closed for modification. We can add new forecast
38     double forecast(ForecastStrategy strategy){
39         return strategy.forecast(data);
40     }
41 }
```

FinancialForecasting.java > Language Support for Java(TM) by Red Hat > FinancialForecast > MovingAverage > forecast(List<Double>)

23 Live Share BLACKBOX Agent Indexing completed. Java: Ready Generative Language Client Open Website Prettier

```
    }  
    }  
}  
  
public class FinancialForecasting {  
    Run main | Debug main | Run | Debug  
    public static void main(String[] args){  
        FinancialForecast.FinancialForecaster forecaster=new FinancialForecast.FinancialForecaster();  
        forecaster.addData(d: 100);  
        forecaster.addData(d: 110);  
        forecaster.addData(d: 120);  
        forecaster.addData(d: 130);  
        forecaster.addData(d: 140);  
        System.out.println("Forecasting with MovingAverage: "+forecaster.forecast(new FinancialForecast.MovingAverage()));  
        System.out.println("Forecasting with LinearTrendForecast: "+forecaster.forecast(new FinancialForecast.LinearTrendForecast()));  
    }  
}
```